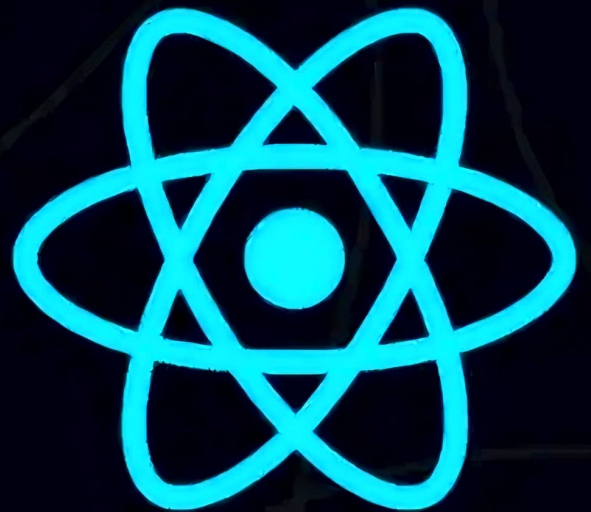


useState **Hook** **in React**



The **useState** hook in React is a fundamental tool for managing state in functional components. It allows you to add **state variables** to your components, enabling them to remember information between renders.

Basic Concept :

State : Data that **changes over time** in your app. For example, user input, API responses, or UI element states.

Hook : **Special functions** in React that let you use state and other features in functional components.

Syntax

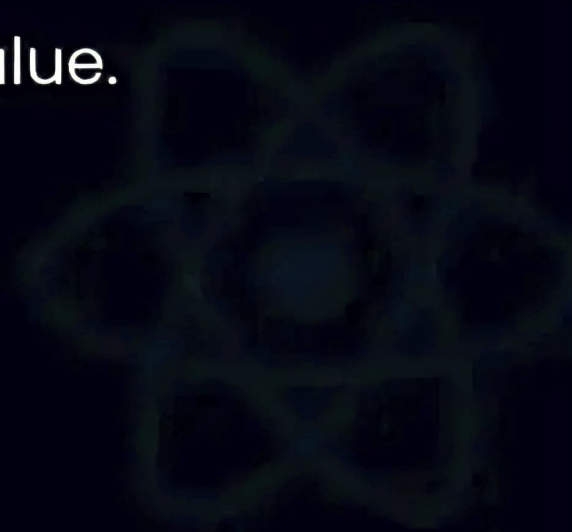


```
const [stateVariable, setStateFunction] = useState(initialValue);
```

stateVariable : The current state value.

setStateFunction : A function to update the state value.

initialValue : The initial state value.



How useState Works

Initialization : You provide an initial state value.

State Variable : React provides a variable to hold the state.

Setter Function : React provides a function to update the state

Lets understand with the simple example

Example 1: Counter

```
import React, { useState } from 'react';

function Counter() {
  // Initialize the count state with an initial value of 0
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

export default Counter;
```

Explanation :

- `const [count, setCount] = useState(0);` initializes the count state to **0**.
- `count` stores the current **state value**.
- **setCount** is a function to update the state.
- Clicking the button increases the count state by 1, which **re-renders the component** with the new value.

Example 2: Toggle Visibility

```
import React, { useState } from 'react';

function ToggleText() {
  const [isVisible, setIsVisible] = useState(true);

  return (
    <div>
      {isVisible && <p>Hello, I am visible!</p>}
      <button onClick={() => setIsVisible(!isVisible)}>
        Toggle Text
      </button>
    </div>
  );
}

export default ToggleText;
```

Explanation :

- `const [isVisible, setIsVisible] = useState(true);` initializes **isVisible** to **true**.
- `isVisible` determines if the text is shown.
- `setIsVisible` toggles the value between `true` and `false`.
- Clicking the button toggles the visibility of the text



Summary:

1. **useState** can hold **any type of value** (number, string, array, object, etc.).
2. The setter function (**setStateFunction**) updates the state and triggers a **re-render**.
3. State updates can be based on the current state using the function form: **setStateFunction(prevState => newState)**.
4. **useState** makes it easy to manage **component state** in a clear and predictable way.